ENTERPRISE DATA MANAGEMENT

ICE ENTERTAINMENT

Submitted by
Parth Prasad Songaonkar

EXECUTIVE SUMMARY

ICE Entertainment is an international entertainment retailer, which offers 500 stores in 10 countries and has a revenue of more than USD 500 million in annual revenue, provided by the sales of digital contents and subscription services. The organisation has had a high level of data availability, but does not have an integrated analytical capability to assess profitability in key business dimensions such as customers, products, stores, time and markets.

This problem is fuelled by disjointed data architecture, with transactional data stored in the Informix Sales System and product and financial data stored in the Oracle ERP System, which restricts the potential of the organisation to create evidence-based insights in a timely manner.

In this report, the proposed enterprise data management solution includes data strategy, dimensional data model, and ETL pipeline to bring together the divergent sources into a single platform to analyze the data. The solution standardises all the financial measures to USD and allows multi-dimensional analysis to be scaled.

The major lessons are that international markets are the most important sources of revenue, the most profitable category is the audio books, and the UK stores are the stable high performers. By creating one source of truth, the solution will provide an opportunity to make decisions based on data and assist in designing a specific customer loyalty program.

DATA STRATEGY CANVAS

<table>
<tr><th>Strategic Objective</th><th colspan="2">Strategic Framing</th><th colspan="2">Strategic Enablement</th></tr>
<tr><td rowspan="2">1.
• Integrate Sales and ERP into a unified USD-based model
• Create a single trusted global data source
• Enable global sales visibility
• Support data-driven decision-making</td><td>2. Key Decisions & Use cases:
• Identify most profitable customers and products
• Analyse store and market performance Support sales and product decisions</td><td>3. AI / Analytics Opportunities:
• Profitability analysis
• Customer segmentation
• Product performance insights
• Predictive analytics (retention)</td><td>5. Data Requirements & Products:
• Sales data + ERP data integration
• Fact table (sales, cost, margin)
• Key dimensions (customer, product, store, time)</td><td>6.Consumers & Users:
• CEO (strategy)
• Sales Manager (performance)
• Purchasing Head (products)
• CRM Team (customers)</td></tr>
<tr><td colspan="2">4. Value Measures & ROI / CBA
• Faster and accurate reporting
• Improved margin visibility
• Better decision-making
• Increased profitability</td><td colspan="2">7. Technology & Systems:
• Informix + Oracle (sources)
• Pentaho (ETL)
• MySQL (data warehouse)</td></tr>
<tr><th colspan="5">Strategic Readiness</th></tr>
<tr><td>8. Portfolio Management
• Enterprise analytics initiative
• Supports multiple business functions
• Enables future analytics capabilities</td><td colspan="2">9. Operating Model & Governance
• Standard definitions (sales, cost, margin)
• USD currency standardisation
• ETL data validation</td><td colspan="2">10. Risks & Ethics
• Data inconsistency
• Currency conversion errors
• Data quality issues
• Privacy and security risks</td></tr>
</table>

Figure 1: Data Strategy Canvas

The Data Strategy Canvas (Figure 1) offers a systematic system that can help ICE Entertainment to resolve its central dilemma of fragmented data in its Sales and ERP systems. The main aim is to determine one USD-standardised source of truth by combining both systems into one analytical platform where one can conduct consistent global profitability analysis.

Key business decisions (identifying profitable customers, products, stores, and markets) are matched with the targeted analytics capabilities (profitability analysis, customer segmentation, and predictive retention modelling) on the canvas. This makes the strategy directly connected to the management priorities.

This architecture proposal focuses on a dimensional model, which will be backed by automated ETL pipeline, facilitating efficient and reliable integration of data. Value is provided in the form of increased accuracy in reporting, increased visibility of marginal, and better decision-making.

Data consistency, risk management, and quality are ensured by governance mechanisms, which will enable ICE Entertainment to grow expandably and with data-driven future analytics initiatives.

DATA FLOW ARCHITECTURE

Figure 2 below depicts an end-to-end data flow architecture with five layers that sequentially process raw operational data to actionable business intelligence depending on the Eckerson Analytical Leaders model.

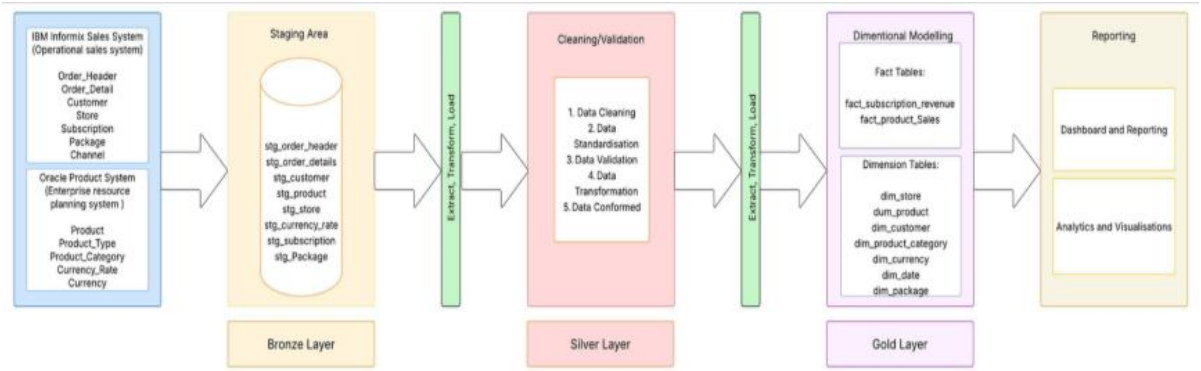


Figure 2: Data Flow Architecture

The data flow architecture will be developed as a layered pipeline to connect Sales and ERP systems of ICE Entertainment to a single platform of analysis.

At the source layer, the transactional information of the Informix Sales System and product and financial information of the Oracle ERP System are pulled off. These data sets are loaded into a Bronze staging layer, raw data is stored in MySQL without any transformation, allowing auditability and being able to recover the data.

Silver layer uses data standardisation, surrogate key generation as well as transformation logic to produce clean and conformed dimension tables. Deriving analytical attributes (e.g. age groups) and SCD Type 2 implementation of both customer and market dimensions are key processes in maintaining historical accuracy.

The Gold layer creates business-ready facts tables, which incorporate sales and subscription data and standardise all monetary figures to USD with monthly exchange rates. Measures like revenue, cost and margin are pre-calculated allowing efficient analysis (Finkelstein et al., 1988).

Lastly, the reporting layer assists SQL-based analytics in line with major business inquiries. In general, the architecture makes sure that there is a single source of truth, enables multi-dimensional analysis, and allows making timely and accurate decisions that are scalable and data driven.

DIMENSIONAL MODEL DESIGN

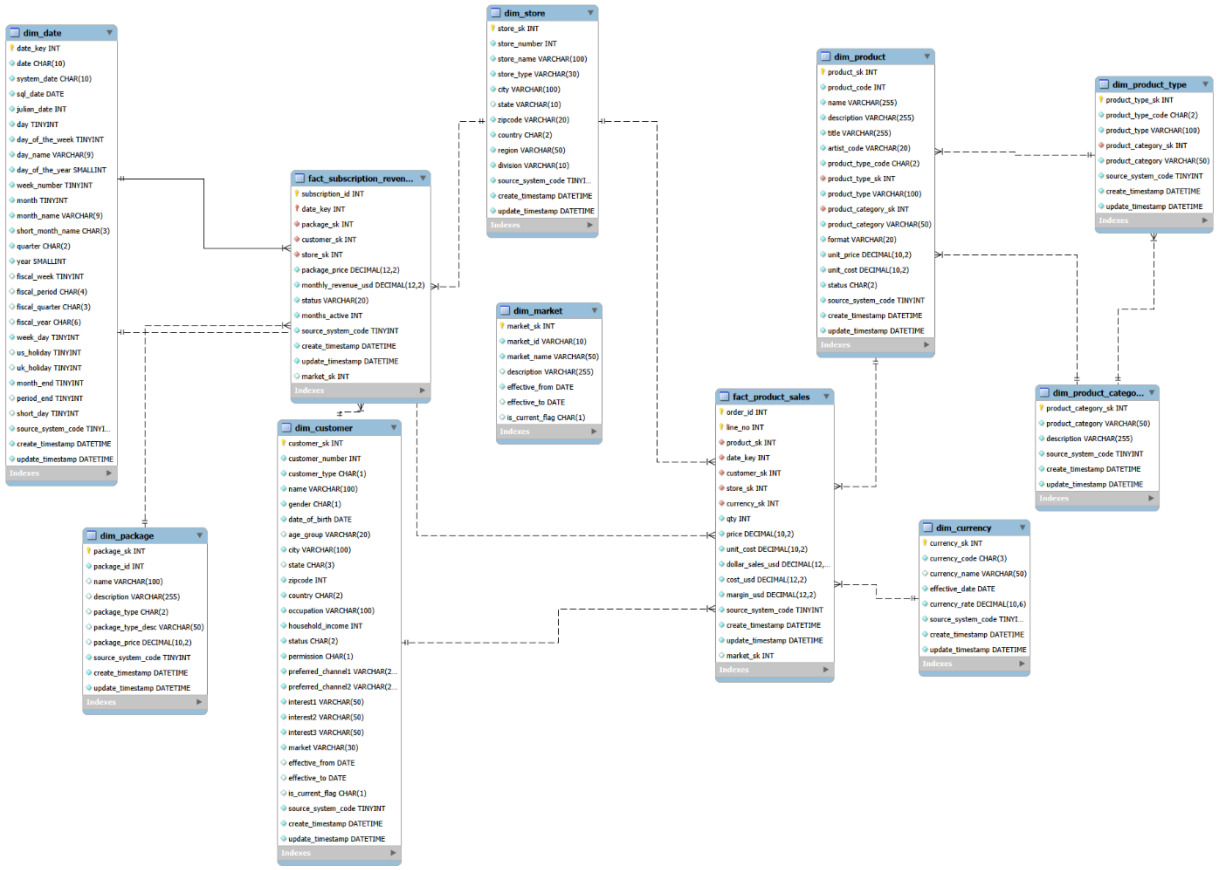


Figure 3: Dimension Model Design

The Kimball approach with a star schema in MySQL data warehouse (dw) was used to develop the dimensional model (Figure 3) of ICE Entertainment. It is comprised of two fact tables and a few dimension tables that are optimised to run well with analytical performance, ease of querying and multi-dimensional analysis (Firestone, n.d.).

Grain Definition

The grain of the fact table is the amount of detail that is stored in the fact table, fact_product_sales is defined at order line item grain (order id + line no) where each row represents a single product sold as a part of an order with the ability to aggregate transactions at the individual order level to global analyses.

fact_subscription_revenue uses subscription grain (subscription_id) containing recurring revenue, duration and status to enable retention and market analysis.

Fact Tables and Measures

The model has two fact tables including the fact_product_sales that holds the quantity, price and unit cost with dollar sales, cost and margin that has been pre-calculated in ETL with monthly exchange rates that makes it consistent without complexities of runtime conversion.

fact subscription revenue stores package price, revenue, duration, and status that can be used to analyze recurring revenue streams. Calculated USD measures in advance promote comparability with the global market, as well as query performance.

Dimension Tables and Attributes

- dim_date contains calendar and fiscal attributes supporting time based analysis aligned to the ICE September to August fiscal year.
- dim_product provides a three-level hierarchy (category, type, title) enabling product drill down analysis. dim_customer stores demographic attributes including derived age_group and implements SCD Type 2 to maintain historical customer location and market at time of sale.
- dim_store enables geographic analysis through a six level hierarchy from division to individual store.
- dim_currency supports multi-currency conversion using monthly exchange rates.
- dim_market is a separate SCD Type 2 dimension supporting changing market definitions without affecting historical reporting.
- dim_package supports subscription analysis.
- dim_date, dim_customer and dim_store are conformed dimensions shared across both fact tables enabling integrated analysis of product sales and subscription revenue (Jovanovic et al., 2012).

Relationships and Model Justification

Dimension tables are linked to fact tables through surrogate keys that provide consistency and decoupling with source systems. The star schema reduces the complexity of joins and allows the creation of useful analytical queries. SCD Type 2 provides historical accuracy of customer and market attributes directly related to the case study needs. Separating dim market and dim customer enables changes in the market to be monitored without modifying large amount of customer data.

Addressing Business Problems

The dimensional model addresses ICE Entertainment fragmented data by combining Sales and ERP systems into a single analytical platform.

Q1 identifies high value customers using combined product and subscription data with demographic and time-based insights.

Q2 provides product hierarchy profitability analysis supporting data driven purchasing decisions.

Q3 enables standardised global store performance comparison using USD metrics and hierarchical analysis.

Q4 supports calendar and fiscal time analysis for operational and strategic planning.

Q5 provides market level analysis with SCD Type 2 preserving historical market definitions.

Enabling Decision Making

The model enhances decision making in four aspects. It brings about a single source of truth in order to have consistent Organisational reporting. It allows the analysis of all dimensions to trace the root causes. It helps in cross stream analysis using conformed dimensions. It is historically sound with SCD Type 2.

DIMENSIONAL MODEL IMPLEMENTATION

The conceptual design was able to identify two fundamental business processes, which consisted of product purchasing and subscription revenue. Both Sales and ERP systems like Customer, Product, Store, Date, Currency, Package and Market were used to derive key business entities. The relationships determine that the customers purchase products via stores on a particular date and subscribe to packages that produce recurrent income.

This schema was modeled as a Kimball star schema and two fact tables, fact product sales was defined at order line item grain with transactional granularity and fact subscription revenue records recurring revenue at subscription level. Integrated cross-process analysis is made possible by conformed dimensions dim_date, dim_customer and dim_store. Dim customer and dim market were subjected to SCD Type 2 to provide historical accuracy and longitudinal analysis (Bonnet, P. (n.d.)).

Physical Database Design

The MySQL data warehouse dw consolidates all the data in a single analytical repository that removes cross-system dependencies. The schema consists of staging, dimension and fact tables that are scalable and performance friendly.

AUTO_INCREMENT-based surrogate keys eliminate the dependence between the warehouse and changes in the source system and enhance the efficiency of the joins. Fact tables impose referential integrity using a foreign key using a NOT NULL constraint and default value constraint to enhance data quality. An explicit dim date table was created through a stored procedure that created 1,826 records between 2022 and 2026 and had both calendar and fiscal attributes tied to the ICE fiscal year so that all the analyses could be done using the same period basis.

ETL Extraction and Orchestration

Data mining was carried out with Pentaho Data Integration that combined the data in both operational systems through Excel files. The ETL design was designed as a modular design with each table having its own transformation to enhance maintainability and debugging.

A single job ICE_Q2_ETL_Job orchestrates all transformations using dependency logic. Before fact tables are loaded, dimension tables are loaded; all surrogate keys have to be available to resolve their lookup. This ensures that the data is consistent and it offers one point of execution of the entire pipeline.

Transformation and Loading

Data standardisation, data type alignment and derived attribute enrichment were all part of transformation. All financial measures were converted to USD using monthly exchange rate to

make it globally comparable.

The fact_product_sales transformation joins the order header and detail data in a Merge Join and the subsequent use of several lookups to determine the dimension surrogate keys. Dollar sales (dollar sales usd), cost (cost usd) and margin (margin usd) are already calculated at loading which simplifies queries and increases performance.

To correctly follow historical changes, SCD Type 2 was used to apply dim customer and dim market with the effective date ranges and status flags so that the outputs of the analysis are not altered to show the right context of each transaction. A truncate and reload policy guarantees the data consistency between pipeline runs (Barahama, A. D., & Wardani, R. (2021)).

Data Validation and Quality Assurance

Row count reconciliation, referential integrity checks and measure validation were used to verify data integrity. Dimension tables were verified and fact tables were verified to have a complete set of data and the correct foreign key mappings and proper calculations.

The date dimension generated 1,826 records as anticipated and fact records were tested to be identical with the dimension keys. Such validation measures will guarantee reliability and confidence in any analytical results (Shanker et al., 2006).

ANALYTICAL QUERIES AND INSIGHTS

Q1- Who are the key customers?

- Findings indicate that the International market has the largest group of customers with the highest subscription revenue of USD 9,878.48 and the highest margin of USD 125,893.51.
- The Rest of Australia is moderately contributing and Victoria has the least revenue and profitability among the three segments.
- These findings imply that resources are to be allocated and marketing campaigns made towards international markets. The poor performance of Victoria as a segment offers the strategic chance of targeted improvement efforts to improve the overall organisational performance.

```sql
SELECT
    c.market,
    ROUND(SUM(f.monthly_revenue_usd), 2) AS total_subscription_revenue_usd,
    COUNT(DISTINCT c.customer_number) AS subscribing_customers,
    COUNT(DISTINCT f.subscription_id) AS total_subscriptions,
    SUM(CASE WHEN f.status = 'Active'
        THEN 1 ELSE 0 END) AS active_subscriptions,
    SUM(CASE WHEN f.status = 'Cancelled'
        THEN 1 ELSE 0 END) AS cancelled_subscriptions,
    SUM(CASE WHEN f.status = 'Expired'
        THEN 1 ELSE 0 END) AS expired_subscriptions,
    ROUND(AVG(f.months_active), 1) AS avg_months_active,
    ROUND(AVG(f.package_price), 2) AS avg_package_price,
    ROUND(SUM(f.monthly_revenue_usd) /
          COUNT(DISTINCT c.customer_number), 2) AS subscription_revenue_per_customer

FROM dw.fact_subscription_revenue f
JOIN dw.dim_customer c ON f.customer_sk = c.customer_sk
GROUP BY c.market
ORDER BY total_subscription_revenue_usd DESC;
```

Figure 4.1: Code for insights about key customers (based on subscription revenue).

market	total_subscription_revenue_usd
International	9878.48
Rest of Australia	785.67
Victoria	480.81

Figure 4.2: Output for insights about key customers (based on subscription revenue).

```sql
SELECT c.market,
    ROUND(SUM(f.margin_usd), 2) AS total_margin_usd,
    COUNT(DISTINCT c.customer_number) AS total_customers_reached,
    COUNT(DISTINCT f.order_id) AS total_transactions,
    SUM(CASE WHEN p.product_category = 'Music'
        THEN f.qty ELSE 0 END) AS music_units_sold,
    SUM(CASE WHEN p.product_category = 'Films'
        THEN f.qty ELSE 0 END) AS films_units_sold,
    SUM(CASE WHEN p.product_category = 'Audio Books'
        THEN f.qty ELSE 0 END) AS audiobooks_units_sold,
    SUM(f.qty) AS total_units_sold,
    ROUND(SUM(f.dollar_sales_usd), 2) AS total_sales_usd,
    ROUND(SUM(f.dollar_sales_usd) / COUNT(DISTINCT c.customer_number), 2) AS revenue_per_customer,
    ROUND(SUM(f.qty) / COUNT(DISTINCT c.customer_number), 2) AS units_per_customer,
    ROUND(COUNT(DISTINCT f.order_id) / COUNT(DISTINCT c.customer_number), 2) AS orders_per_customer
FROM dw.fact_product_sales f
JOIN dw.dim_customer c  ON f.customer_sk = c.customer_sk
JOIN dw.dim_product p   ON f.product_sk  = p.product_sk
GROUP BY c.market
ORDER BY total_margin_usd DESC;
```

Figure 4.3: Code for insights about key customers (based on margin).

market	total_margin_usd
International	125893.51
Rest of Australia	9082.91
Victoria	6613.36

Figure 4.4: Output for insights about key customers (based on margin).

Q2-Which products are the most profitable?

- The types of products that are most profitable are determined in this analysis to inform product investment and strategy.
- Results indicate that the most profitable in absolute terms with the highest total margin of USD 52,427.74 are the Audio Books and then Films. Music shows the greatest percentage of margin but with a relatively lower total margin even with high order volume - a major difference that the management should look into when comparing the performance of the categories.
- These results form a clear basis of prioritisation. The most important area to maximise absolute revenue and profit should be on the Audio Books. Music is one of the efficiency opportunities that the organisation should capitalise. Further drilling down to most lucrative types of Audio Books products will contribute to highest profitability.

```
SELECT
    p.product_category,
    ROUND(SUM(f.margin_usd), 2) AS total_margin_usd,
    ROUND(SUM(f.margin_usd) / SUM(f.dollar_sales_usd) * 100, 2) AS margin_pct,
    COUNT(DISTINCT f.order_id) AS total_orders,
    SUM(f.qty) AS total_units_sold,
    ROUND(SUM(f.dollar_sales_usd), 2) AS total_sales_usd,
    ROUND(SUM(f.cost_usd), 2) AS total_cost_usd,
    ROUND(SUM(f.margin_usd) / (SELECT SUM(margin_usd)
        FROM dw.fact_product_sales) * 100, 2) AS pct_of_total_margin
FROM dw.fact_product_sales f
JOIN dw.dim_product p ON f.product_sk = p.product_sk
GROUP BY p.product_category
ORDER BY total_margin_usd DESC;
```

Figure 5.1: Code for most profitable product category (based on total margin).

product_category	total_margin_usd	margin_pct	total_orders
Audio Books	52427.74	45.16	9871
Films	51384.87	37.24	9763
Music	37777.17	58.91	10148

Figure 5.2: Output for most profitable product category (based on total margin).

```sql
SELECT
    p.product_category, p.product_type, p.title,
    d.year AS calendar_year,
    CASE
        WHEN d.month IN (12, 1, 2)  THEN 'Summer'
        WHEN d.month IN (3, 4, 5)   THEN 'Autumn'
        WHEN d.month IN (6, 7, 8)   THEN 'Winter'
        WHEN d.month IN (9, 10, 11) THEN 'Spring'
    END AS season,
    d.month_name AS calendar_month,
    SUM(f.qty) AS unit_sales,
    ROUND(SUM(f.dollar_sales_usd), 2) AS dollar_sales_usd,
    ROUND(SUM(f.cost_usd), 2) AS cost_usd,
    ROUND(SUM(f.margin_usd), 2) AS margin_usd,
    ROUND(SUM(f.margin_usd) / SUM(f.dollar_sales_usd) * 100, 2)     AS margin_pct
FROM dw.fact_product_sales f JOIN dw.dim_product p ON f.product_sk = p.product_sk
JOIN dw.dim_date d ON f.date_key = d.date_key
GROUP BY p.product_category, p.product_type, p.title, d.year, d.month, d.month_name
ORDER BY margin_usd DESC
LIMIT 20;
```

Figure 5.3: Code for most profitable product category (based on product type).

product_category	product_type	title
Audio Books	Traditional Bk	Code Business
Audio Books	HR Books	Principles Change
Audio Books	Engineering Bk	Wisdom Success
Audio Books	Cooking Books	Way Growth

Figure 5.4: Output for most profitable product category (based on product type).

Q3- Which store locations are the most profitable?

- The analysis identifies the most profitable stores in terms of their margin percentage in order to facilitate location based performance measurement and tactical decision making.

- Findings indicate that Naples20 in Italy under the EMEA division is the most profitable store of the overall with the highest margin percentage of 50.01 indicating good performance of the EMEA region. Among Australian stores Perth6 is the best performer with a margin percentage of 48.90 and several other Perth stores also feature as the best stores in the country.

- These results point to two strategic areas already performing well in terms of results, EMEA and Perth. The potential to repeat what is working in low performing stores and still take advantage of the high performing stores to push profitability at the

organisational level is evident.

```
SELECT
    s.store_name,
    s.store_type,
    s.division,
    s.region,
    ROUND(SUM(f.margin_usd) / SUM(f.dollar_sales_usd) * 100, 2) AS margin_pct,
    s.country, s.state, s.city, d.year AS calendar_year,
    d.month_name AS calendar_month,
    SUM(f.qty) AS unit_sales,
    ROUND(SUM(f.dollar_sales_usd), 2) AS dollar_sales_usd,
    ROUND(SUM(f.cost_usd), 2) AS cost_usd,
    ROUND(SUM(f.margin_usd), 2) AS margin_usd
FROM dw.fact_product_sales f JOIN dw.dim_store s ON f.store_sk = s.store_sk
JOIN dw.dim_date d  ON f.date_key = d.date_key
GROUP BY
    s.store_name, s.store_type, s.division, s.region, s.country, s.state, s.city,
    d.year,
    d.month,
    d.month_name
ORDER BY dollar_sales_usd DESC
LIMIT 20;
```

Figure 6.1: Code for most profitable store locations (globally).

store_name	store_type	division	region	margin_pct
Naples20	Distribution Centre	EMEA	Italy	50.01
Liverpool20	Mini Outlet	EMEA	United Kingdom	47.53
Bradford16	Full Outlet	EMEA	United Kingdom	46.50
Leeds18	Mini Outlet	EMEA	United Kingdom	46.23

Figure 6.2: Output for most profitable store locations (globally).

```sql
SELECT
    s.store_name, s.store_type,
    ROUND(SUM(f.margin_usd) / SUM(f.dollar_sales_usd) * 100, 2) AS margin_pct,
    s.division, s.region, s.country, s.state, s.city,
    d.year AS calendar_year,
    d.month_name AS calendar_month,
    SUM(f.qty) AS unit_sales,
    ROUND(SUM(f.dollar_sales_usd), 2) AS dollar_sales_usd,
    ROUND(SUM(f.cost_usd), 2) AS cost_usd,
    ROUND(SUM(f.margin_usd), 2) AS margin_usd
FROM dw.fact_product_sales f
JOIN dw.dim_store s ON f.store_sk = s.store_sk
JOIN dw.dim_date d  ON f.date_key = d.date_key
WHERE s.country = 'AU'
GROUP BY
    s.store_name, s.store_type, s.division, s.region, s.country, s.state, s.city,
    d.year, d.month, d.month_name
ORDER BY margin_usd DESC
LIMIT 20;
```

Figure 6.3: Code for most profitable store locations (in Australia).

store_name	store_type	margin_pct	division	region
Perth6	Distribution Centre	48.90	AAA	Australia
Perth12	Full Outlet	47.26	AAA	Australia
Sydney13	Full Outlet	39.17	AAA	Australia
Adelaide10	Full Outlet	44.17	AAA	Australia
Canberra15	Mini Outlet	39.47	AAA	Australia

Figure 6.4: Output for most profitable store locations (in Australia).

Q4- Which time periods are the most profitable?

- The analysis helps in determining the most profitable times to assist seasonal planning and performance measurement.
- Results indicate that January was the most profitable month with the highest sales of USD 32,489.14 and high margin percentage. The highest sales of USD 60,822.75 is registered in Q1 on a quarterly basis showing a good year beginning. The percentage of margin is not significantly varied in all the four quarters implying that profitability differences are influenced by volume and not efficiency.
- The findings allow the organisation to target marketing campaigns, promotions and inventory planning on the high performing months like January and Q1 and optimise the use of seasonal patterns to make the most of the resource utilisation during the low activity periods.

```sql
SELECT
    d.month_name AS calendar_month,
    ROUND(SUM(f.dollar_sales_usd), 2) AS dollar_sales_usd,
    ROUND(SUM(f.margin_usd) / SUM(f.dollar_sales_usd) * 100, 2) AS margin_pct,
    d.month AS month_number,
    d.fiscal_period,
    CASE
        WHEN d.month IN (12, 1, 2)  THEN 'Summer'
        WHEN d.month IN (3, 4, 5)   THEN 'Autumn'
        WHEN d.month IN (6, 7, 8)   THEN 'Winter'
        WHEN d.month IN (9, 10, 11) THEN 'Spring'
    END AS season,
    COUNT(DISTINCT f.order_id) AS total_orders,
    ROUND(SUM(f.margin_usd), 2) AS margin_usd,
    SUM(f.qty) AS unit_sales,
    ROUND(SUM(f.cost_usd), 2) AS cost_usd
FROM dw.fact_product_sales f JOIN dw.dim_date d ON f.date_key = d.date_key
GROUP BY
    d.month, d.month_name, d.fiscal_period
ORDER BY margin_usd DESC;
```

Figure 7.1: Code for most profitable month.

	calendar_month	dollar_sales_usd	margin_pct
▶	January	32489.14	44.51
	July	26963.07	44.21
	February	26453.88	44.37
	April	25828.63	44.43

Figure 7.2: Output for most profitable month.

```sql
SELECT
    d.quarter AS calendar_quarter,
    d.fiscal_quarter,
    ROUND(SUM(f.dollar_sales_usd), 2) AS dollar_sales_usd,
    ROUND(SUM(f.margin_usd) / SUM(f.dollar_sales_usd) * 100, 2) AS margin_pct,
    COUNT(DISTINCT f.order_id) AS total_orders,
    COUNT(DISTINCT d.month) AS months_in_quarter,
    SUM(f.qty) AS unit_sales,
    ROUND(SUM(f.cost_usd), 2) AS cost_usd,
    ROUND(SUM(f.margin_usd), 2) AS margin_usd,
    ROUND(SUM(f.dollar_sales_usd) / COUNT(DISTINCT d.month), 2) AS avg_monthly_sales_usd,
    ROUND(SUM(f.margin_usd) / COUNT(DISTINCT d.month), 2) AS avg_monthly_margin_usd
FROM dw.fact_product_sales f JOIN dw.dim_date d ON f.date_key = d.date_key
WHERE d.fiscal_quarter IS NOT NULL
GROUP BY
    d.quarter,
    d.fiscal_quarter
ORDER BY margin_usd DESC;
```

Figure 7.3: Code for most profitable quarter.

calendar_quarter	fiscal_quarter	dollar_sales_usd	margin_pct	total_orders
Q1	FQ2	60822.75	44.42	2901
Q4	FQ1	52209.30	44.33	2396
Q2	FQ3	51556.78	44.49	2404
Q3	FQ4	50763.27	44.56	2451

Figure 7.4: Output for most profitable quarter.

Q5- Which market is the most profitable?

- This analysis helps in determining the most profitable market to aid in strategic decisions in relation to market focus and resource allocation.

- Unexpected outcomes indicate that the International market is the most lucrative that makes the highest sales of USD 282,929.92 and margin USD 125,893.51 by a large margin as compared to Rest of Australia and Victoria. Though the margin percentages are comparable in all the three markets, the International market has a very high sales volume which contributes to its high profitability.

- These results suggest that the most direct way of maximising returns is through international expansion and investment. The rising domestic market performance can be another opportunity to improve the overall organisational profitability.

```sql
SELECT
    c.market,
    COUNT(DISTINCT c.customer_number) AS total_customers,
    COUNT(DISTINCT f.order_id) AS total_orders,
    SUM(f.qty) AS unit_sales,
    ROUND(SUM(f.dollar_sales_usd), 2) AS dollar_sales_usd,
    ROUND(SUM(f.cost_usd), 2) AS cost_usd,
    ROUND(SUM(f.margin_usd), 2) AS margin_usd,
    ROUND(SUM(f.margin_usd) /
          SUM(f.dollar_sales_usd) * 100, 2)     AS margin_pct,
    ROUND(SUM(f.dollar_sales_usd) /
          COUNT(DISTINCT c.customer_number), 2) AS revenue_per_customer,
    ROUND(SUM(f.margin_usd) /
          COUNT(DISTINCT c.customer_number), 2) AS margin_per_customer
FROM dw.fact_product_sales f
JOIN dw.dim_customer c ON f.customer_sk = c.customer_sk
GROUP BY c.market
ORDER BY margin_usd DESC;
```

Figure 8.1: Code for most profitable market.

market	total_customers	total_orders	unit_sales	dollar_sales_usd	cost_usd	margin_usd	margin_pct
International	4227	13426	84737	282929.97	157041.70	125893.51	44.50
Rest of Australia	312	983	6409	20396.98	11315.10	9082.91	44.53
Victoria	216	673	4327	14895.11	8282.21	6613.36	44.40

Figure 8.2: Output for most profitable market.

Conclusion

The dimensional model that is created in reference to the ICE Entertainment directly responds to one of the most urgent data problems the organisation faces the fragmentation of information in two dissimilar source systems. The solution provides a consistent reliable platform to business intelligence by consolidating these into a single analytical platform. The Kimball star schema, Pentaho ETL pipeline and Bronze-Silver-Gold architecture work together to provide pre-calculated USD measures, SCD Type 2 history tracking and conformed dimensions, all of which collectively answer all of the five most important business questions of the organisation. Bigger picture, this gives the leadership at ICE Entertainment what they have been long overdue a reliable, consistent data environment to support evidence-based decision making and achieve the customer segmentation ability needed to roll out the first loyalty program in the organisation.

References

1. Firestone, J. (n.d.). Dimensional modeling and E-R modeling in the data warehouse. *Executive Information Systems, Inc*. Dimensional Modeling and E-R Modeling in the Data Warehouse

2. Bonnet, P. (n.d.) Enterprise data Governance: Reference and master data management semantic modeling. *Wiley Enterprise Data Governance: Reference and Master Data Management Semantic ... - Pierre Bonnet*

3. Barahama, A. D., & Wardani, R. (2021). Utilization extract, transform, load for developing data warehouse in education using Pentaho data integration. *Journal of Physics Conference Series*, *2111*(1), 012030. https://doi.org/10.1088/1742-6596/2111/1/012030

4. Shanker, U., Misra, M., Sarje, A. K., & Department of Electronics & Computer Engineering, Indian Institute of Technology Roorkee. (2006). Some performance issues in distributed real time database systems. In *Proceedings of the VLDB 2006 PhD Workshop*. http://ftp.informatik.rwth-aachen.de/Publications/CEUR-WS/Vol-170/paper6.pdf

5. Finkelstein, S., Schkolnick, M., & Tiberio, P. (1988). Physical database design for relational databases. *ACM Transactions on Database Systems*, *13*(1), 91–128. https://doi.org/10.1145/42201.42205

6. Jovanovic, V., Bojicic, I., Knowles, C., & Pavlic, M. (2012). PERSISTENT STAGING AREA MODELS FOR DATA WAREHOUSES. *Issues in Information Systems*. https://doi.org/10.48009/1_iis_2012_121-132

Appendices

Appendix 1: Data Dictionary

FACT TABLES

fact_product_sales
Grain: One row per order line item (order_id + line_no) | Rows: 44,443

Column Name	Data Type	Description
order_id	VARCHAR	Order reference number from Sales System
line_no	INT	Line number within the order
date_key	INT (FK)	Links to dim_date — when the sale occurred
product_sk	INT (FK)	Links to dim_product — what was sold
customer_sk	INT (FK)	Links to dim_customer — who bought it (SCD Type 2)
store_sk	INT (FK)	Links to dim_store — where it was sold
currency_sk	INT (FK)	Links to dim_currency — currency and exchange rate
market_sk	INT (FK)	Links to dim_market — market segment at time of sale
qty	INT	Number of units sold on this line item
price	DECIMAL(10,2)	Selling price per unit in local currency
unit_cost	DECIMAL(10,2)	Cost price per unit in local currency
dollar_sales_usd	DECIMAL(10,2)	Pre-calculated: qty x price x monthly exchange rate
cost_usd	DECIMAL(10,2)	Pre-calculated: qty x unit_cost x monthly exchange rate
margin_usd	DECIMAL(10,2)	Pre-calculated: dollar_sales_usd minus cost_usd

fact_subscription_revenue
Grain: One row per subscription (subscription_id) | Rows: 504

Column Name	Data Type	Description
subscription_id	VARCHAR	Unique subscription reference number
date_key	INT (FK)	Links to dim_date — subscription start date
package_sk	INT (FK)	Links to dim_package — which package was subscribed to
customer_sk	INT (FK)	Links to dim_customer — who subscribed (SCD Type 2)
store_sk	INT (FK)	Links to dim_store — which store sold the subscription
market_sk	INT (FK)	Links to dim_market — market segment of subscriber
package_price	DECIMAL(10,2)	Monthly price of the subscription package in USD
monthly_revenue_usd	DECIMAL(10,2)	Monthly recurring revenue generated in USD
months_active	INT	Number of months subscription was active
status	VARCHAR(20)	Current subscription state: Active, Cancelled or Expired

DIMENSION TABLES

dim_date
Grain: One row per calendar day | Rows: 1,826

Column Name	Data Type	Description
date_key	INT (PK)	Surrogate key — auto-generated integer
sql_date	DATE	Full calendar date value
year	INT	Calendar year: 2022 to 2026
quarter	VARCHAR	Calendar quarter: Q1 Q2 Q3 Q4
month	INT	Month number: 1 to 12
month_name	VARCHAR	Month name: January to December
week_number	INT	Week number within the year
day_name	VARCHAR	Day name: Monday to Sunday
week_day	TINYINT	0 = weekend, 1 = weekday
fiscal_year	VARCHAR	ICE fiscal year: FY2022 to FY2026 (Sep to Aug)
fiscal_quarter	VARCHAR	ICE fiscal quarter: FQ1 to FQ4
fiscal_period	VARCHAR	ICE fiscal month: FP01 to FP12

dim_product
Grain: One row per product | Rows: 1,000

Column Name	Data Type	Description
product_sk	INT (PK)	Surrogate key — auto-generated integer
product_id	VARCHAR	Natural key from Oracle ERP System
product_category	VARCHAR	Highest hierarchy level: Music, Films, Audio Books
product_type	VARCHAR	Mid hierarchy: Classical Music, Drama Films etc
title	VARCHAR	Individual product title — lowest hierarchy level
format	VARCHAR	Physical format: CD, DVD, Digital, Vinyl etc
artist_code	VARCHAR	Artist or author reference code
unit_price	DECIMAL(10,2)	Standard selling price in local currency
unit_cost	DECIMAL(10,2)	Standard cost price in local currency

dim_product_category
Grain: One row per product category | Rows: 3

Column Name	Data Type	Description
category_sk	INT (PK)	Surrogate key
category_id	VARCHAR	Natural key from ERP System
category_name	VARCHAR	Music, Films or Audio Books

dim_product_type
Grain: One row per product type | Rows: 65

Column Name	Data Type	Description
type_sk	INT (PK)	Surrogate key
type_id	VARCHAR	Natural key from ERP System
type_name	VARCHAR	Product type name e.g. Classical Music, Drama Films

category_sk	INT (FK)	Links to dim_product_category

dim_customer [SCD Type 2]
Grain: One row per customer per market/address change | Rows: 5,000

Column Name	Data Type	Description
customer_sk	INT (PK)	Surrogate key — unique per SCD Type 2 version
customer_number	VARCHAR	Natural key from Informix Sales System
name	VARCHAR	Customer full name
date_of_birth	DATE	Used to derive age_group during ETL
age_group	VARCHAR	Derived: <=30 or 31-50 or 50+ (calculated in Pentaho)
gender	VARCHAR	Customer gender: Male or Female
occupation	VARCHAR	Customer occupation
household_income	VARCHAR	Income bracket
zipcode	VARCHAR	Postcode at time of this SCD record
city	VARCHAR	City at time of this SCD record
country	VARCHAR	Country code e.g. AU GB DE IN
market	VARCHAR	Victoria, Rest of Australia or International
preferred_channel1	VARCHAR	Primary preferred shopping channel
preferred_channel2	VARCHAR	Secondary preferred channel
interest1	VARCHAR	Customer interest 1
interest2	VARCHAR	Customer interest 2
interest3	VARCHAR	Customer interest 3
effective_from	DATE	SCD Type 2: date this record became active
effective_to	DATE	SCD Type 2: date this record was superseded
is_current_flag	CHAR(1)	SCD Type 2: Y = current record, N = historical

dim_store
Grain: One row per store | Rows: 500

Column Name	Data Type	Description
store_sk	INT (PK)	Surrogate key
store_id	VARCHAR	Natural key from Sales System
store_name	VARCHAR	Individual store name e.g. Melbourne1, London6
store_type	VARCHAR	Full Outlet, Mini Outlet or Distribution Centre
division	VARCHAR	Highest hierarchy: EMEA or AAA
region	VARCHAR	Second level: United Kingdom, Germany, Australia etc
country	VARCHAR	Country code: GB DE AU IN CN ZA JP
state	VARCHAR	Australian stores only — NULL for all other countries
city	VARCHAR	City name e.g. Melbourne, London, Dusseldorf

dim_currency

Grain: One row per currency per month | Rows: 369

Column Name	Data Type	Description
currency_sk	INT (PK)	Surrogate key
currency_code	VARCHAR	ISO code: AUD EUR GBP INR JPY ZAR CNY
currency_name	VARCHAR	Full currency name e.g. Australia, Dollars
effective_date	DATE	Month for which this exchange rate applies
currency_rate	DECIMAL(10,6)	Monthly rate to convert local currency to USD

dim_package
Grain: One row per subscription package | Rows: 12

Column Name	Data Type	Description
package_sk	INT (PK)	Surrogate key
package_id	VARCHAR	Natural key from Sales System
name	VARCHAR	Package name e.g. Music123, FilmFlex
package_type	VARCHAR	Type code: GM ST TG RT TP SM
package_type_desc	VARCHAR	General Mix, Single Type, Together, Retention, Trial
package_price	DECIMAL(10,2)	Monthly subscription price in USD

dim_market [SCD Type 2]
Grain: One row per market segment per boundary definition | Rows: 3

Column Name	Data Type	Description
market_sk	INT (PK)	Surrogate key — unique per SCD Type 2 version
market_id	VARCHAR	Natural identifier: VIC, ROA, INTL
market_name	VARCHAR	Victoria, Rest of Australia or International
description	VARCHAR	Business definition of the market boundary
effective_from	DATE	SCD Type 2: date this market definition became active
effective_to	DATE	SCD Type 2: date this definition was superseded
is_current_flag	CHAR(1)	SCD Type 2: Y = current definition, N = historical

STAGING TABLES

Table Name	Source File	Purpose
stg_order_header	Order_Header.xlsx	Raw order header data from Informix Sales System
stg_order_details	Order_Details.xlsx	Raw order line item data from Informix Sales System
stg_customer	Customer.xlsx	Raw customer demographic data from Sales System
stg_product	Product.xlsx	Raw product master data from Oracle ERP System
stg_store	Store.xlsx	Raw store location data from Sales System
stg_currency_rate	Currency_Rate.xlsx	Raw monthly exchange rates from Oracle ERP System
stg_subscription	Subscription.xlsx	Raw subscription records from Sales System
stg_package	Package.xlsx	Raw package definitions from Sales System

Appendix 2: SQL Implementation

1. Creating database dw and dimension table – dim_product_category

```
CREATE DATABASE IF NOT EXISTS dw;
USE dw;

DROP TABLE IF EXISTS dw.dim_product_category;

CREATE TABLE dw.dim_product_category (
    product_category_sk     INT             NOT NULL AUTO_INCREMENT,
    product_category        VARCHAR(50)     NOT NULL,
    description             VARCHAR(255)    NOT NULL,
    source_system_code      TINYINT         NOT NULL DEFAULT 0,
    create_timestamp        DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP,
    update_timestamp        DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT pk_dim_product_category PRIMARY KEY (product_category_sk),
    CONSTRAINT uq_dim_product_category UNIQUE (product_category)
);
```

2. Creating dimension table – dim_market

```
CREATE TABLE dw.dim_market (
    market_sk       INT AUTO_INCREMENT PRIMARY KEY,
    market_id       VARCHAR(10)  NOT NULL,
    market_name     VARCHAR(50)  NOT NULL,
    description     VARCHAR(255),
    effective_from  DATE         NOT NULL,
    effective_to    DATE,
    is_current_flag CHAR(1)      DEFAULT 'Y'
);

INSERT INTO dw.dim_market
(market_id, market_name, description,
 effective_from, effective_to, is_current_flag)
VALUES
('VIC',  'Victoria',
 'Customers and stores in Victoria Australia',
 '2022-01-01', '9999-12-31', 'Y'),
('ROA',  'Rest of Australia',
 'All other Australian customers and stores',
 '2022-01-01', '9999-12-31', 'Y'),
('INTL', 'International',
```

```
    'All customers and stores outside Australia',
    '2022-01-01', '9999-12-31', 'Y');
ALTER TABLE dw.fact_product_sales
ADD COLUMN market_sk INT AFTER currency_sk;
ALTER TABLE dw.fact_subscription_revenue
ADD COLUMN market_sk INT AFTER store_sk;
SET SQL_SAFE_UPDATES = 0;
UPDATE dw.fact_product_sales f
JOIN dw.dim_customer c
    ON f.customer_sk = c.customer_sk
JOIN dw.dim_market m
    ON c.market = m.market_name
SET f.market_sk = m.market_sk
WHERE m.is_current_flag = 'Y';
UPDATE dw.fact_subscription_revenue f
JOIN dw.dim_customer c
    ON f.customer_sk = c.customer_sk
JOIN dw.dim_market m
    ON c.market = m.market_name
SET f.market_sk = m.market_sk
 WHERE m.is_current_flag = 'Y';

SET SQL_SAFE_UPDATES = 1;
SELECT * FROM dw.dim_market;

SELECT COUNT(*)        AS total_rows,
       COUNT(market_sk) AS rows_with_market_sk
FROM dw.fact_product_sales;

SELECT COUNT(*)        AS total_rows,
       COUNT(market_sk) AS rows_with_market_sk
FROM dw.fact_subscription_revenue;
```

3. Creating dimension table – dim_product_type

```
DROP TABLE IF EXISTS dw.dim_product_type;

CREATE TABLE dw.dim_product_type (
    product_type_sk         INT             NOT NULL AUTO_INCREMENT,
    product_type_code       CHAR(2)         NOT NULL,
    product_type            VARCHAR(100)    NOT NULL,
    product_category_sk     INT             NOT NULL,
    product_category        VARCHAR(50)     NOT NULL,
    source_system_code      TINYINT         NOT NULL DEFAULT 0,
    create_timestamp        DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP,
    update_timestamp        DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT pk_dim_product_type PRIMARY KEY (product_type_sk),
    CONSTRAINT uq_dim_product_type UNIQUE (product_type_code),
    CONSTRAINT fk_dpt_category
        FOREIGN KEY (product_category_sk)
        REFERENCES dw.dim_product_category (product_category_sk)
);
```

4. Creating dimension table – dim_product

```
DROP TABLE IF EXISTS dw.dim_product;

CREATE TABLE dw.dim_product (
    product_sk              INT             NOT NULL AUTO_INCREMENT,
    product_code            INT             NOT NULL,
    name                    VARCHAR(255)    NOT NULL,
    description             VARCHAR(255)    NOT NULL,
    title                   VARCHAR(255)    NOT NULL,
    artist_code             VARCHAR(20)     NOT NULL,
    product_type_code       CHAR(2)         NOT NULL,
    product_type_sk         INT             NOT NULL,
    product_type            VARCHAR(100)    NOT NULL,
    product_category_sk     INT             NOT NULL,
    product_category        VARCHAR(50)     NOT NULL,
    format                  VARCHAR(20)     NOT NULL,
    unit_price              DECIMAL(10,2)   NOT NULL,
    unit_cost               DECIMAL(10,2)   NOT NULL,
    status                  CHAR(2)         NOT NULL,
    source_system_code      TINYINT         NOT NULL DEFAULT 0,
    create_timestamp        DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP,
    update timestamp        DATETIME        NOT NULL DEFAULT CURRENT TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
     CONSTRAINT pk_dim_product PRIMARY KEY (product_sk),
     CONSTRAINT uq_dim_product UNIQUE (product_code),
     CONSTRAINT fk_dp_product_type
         FOREIGN KEY (product_type_sk)
         REFERENCES dw.dim_product_type (product_type_sk),
     CONSTRAINT fk_dp_product_category
         FOREIGN KEY (product_category_sk)
         REFERENCES dw.dim_product_category (product_category_sk)
 );
```

5. Creating dimension table – dim_currency

```
DROP TABLE IF EXISTS dw.dim_currency;

CREATE TABLE dw.dim_currency (
    currency_sk         INT             NOT NULL AUTO_INCREMENT,
    currency_code       CHAR(3)         NOT NULL,
    currency_name       VARCHAR(50)     NOT NULL,
    effective_date      DATE            NOT NULL,
    currency_rate       DECIMAL(10,6)   NOT NULL,
    source_system_code  TINYINT         NOT NULL DEFAULT 0,
    create_timestamp    DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP,
    update_timestamp    DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT pk_dim_currency PRIMARY KEY (currency_sk),
    CONSTRAINT uq_dim_currency UNIQUE (currency_code, effective_date)
);
```

6. Creating dimension table – dim_customer

```
DROP TABLE IF EXISTS dw.dim_customer;

CREATE TABLE dw.dim_customer (
    customer_sk             INT             NOT NULL AUTO_INCREMENT,
    customer_number         INT             NOT NULL,
    customer_type           CHAR(1)         NOT NULL,
    name                    VARCHAR(100)    NOT NULL,
    gender                  CHAR(1)         NOT NULL,
    date_of_birth           DATE            NOT NULL,
    age_group               VARCHAR(10)     NOT NULL,
    city                    VARCHAR(100)    NOT NULL,
    state                   CHAR(3)         NULL,
    zipcode                 INT             NOT NULL,
    country                 CHAR(2)         NOT NULL,
    occupation              VARCHAR(100)    NOT NULL,
    household_income        INT             NOT NULL,
    status                  CHAR(2)         NOT NULL,
    permission              CHAR(1)         NOT NULL,
    preferred_channel1      VARCHAR(20)     NOT NULL,
    preferred_channel2      VARCHAR(20)     NOT NULL,
    interest1               VARCHAR(50)     NOT NULL,
```

```
    preferred_channel2      VARCHAR(20)     NOT NULL,
    interest1               VARCHAR(50)     NOT NULL,
    interest2               VARCHAR(50)     NOT NULL,
    interest3               VARCHAR(50)     NOT NULL,
    market                  VARCHAR(30)     NOT NULL,
    effective_from          DATE            NOT NULL,
    effective_to            DATE            NOT NULL,
    is_current_flag         CHAR(1)         NOT NULL DEFAULT 'Y',
    source_system_code      TINYINT         NOT NULL DEFAULT 0,
    create_timestamp        DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP,
    update_timestamp        DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT pk_dim_customer PRIMARY KEY (customer_sk)
);
```

7. Creating dimension table – dim_store

```
CREATE TABLE dw.dim_store (
    store_sk            INT             NOT NULL AUTO_INCREMENT,
    store_number        INT             NOT NULL,
    store_name          VARCHAR(100)    NOT NULL,
    store_type          VARCHAR(30)     NOT NULL,
    city                VARCHAR(100)    NOT NULL,
    state               CHAR(3)         NULL,
    zipcode             VARCHAR(20)     NOT NULL,
    country             CHAR(2)         NOT NULL,
    region              VARCHAR(50)     NOT NULL,
    division            VARCHAR(10)     NOT NULL,
    source_system_code  TINYINT         NOT NULL DEFAULT 0,
    create_timestamp    DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP,
    update_timestamp    DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT pk_dim_store PRIMARY KEY (store_sk),
    CONSTRAINT uq_dim_store UNIQUE (store_number)
);
```

8. Creating dimension table – dim_package

```
CREATE TABLE dw.dim_package (
    package_sk          INT             NOT NULL AUTO_INCREMENT,
    package_id          INT             NOT NULL,
    name                VARCHAR(100)    NOT NULL,
    description         VARCHAR(255)    NOT NULL,
    package_type        CHAR(2)         NOT NULL,
    package_type_desc   VARCHAR(50)     NOT NULL,
    package_price       DECIMAL(10,2)   NOT NULL,
    source_system_code  TINYINT         NOT NULL DEFAULT 0,
    create_timestamp    DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP,
    update_timestamp    DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT pk_dim_package PRIMARY KEY (package_sk),
    CONSTRAINT uq_dim_package UNIQUE (package_id)
);
```

9. Creating fact table Fact_product_sale

```sql
CREATE TABLE dw.fact_product_sales (
    order_id            INT             NOT NULL,
    line_no             INT             NOT NULL,
    product_sk          INT             NOT NULL,
    date_key            INT             NOT NULL,
    customer_sk         INT             NOT NULL,
    store_sk            INT             NOT NULL,
    currency_sk         INT             NOT NULL,
    qty                 INT             NOT NULL,
    price               DECIMAL(10,2)   NOT NULL,
    unit_cost           DECIMAL(10,2)   NOT NULL,
    dollar_sales_usd    DECIMAL(12,2)   NOT NULL,
    cost_usd            DECIMAL(12,2)   NOT NULL,
    margin_usd          DECIMAL(12,2)   NOT NULL,
    source_system_code  TINYINT         NOT NULL DEFAULT 0,
    create_timestamp    DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP,
    update_timestamp    DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT pk_fact_product_sales PRIMARY KEY (order_id, line_no),
    CONSTRAINT fk_fps_product
        FOREIGN KEY (product_sk)
        REFERENCES dw.dim_product (product_sk),
    CONSTRAINT fk_fps_date
        FOREIGN KEY (date_key)
        REFERENCES dw.dim_date (date_key),
    CONSTRAINT fk_fps_customer
        FOREIGN KEY (customer_sk)
        REFERENCES dw.dim_customer (customer_sk),
    CONSTRAINT fk_fps_store
        FOREIGN KEY (store_sk)
        REFERENCES dw.dim_store (store_sk),
    CONSTRAINT fk_fps_currency
        FOREIGN KEY (currency_sk)
        REFERENCES dw.dim_currency (currency_sk)
);
```

10. Creating fact table Fact_subsciption

```sql
CREATE TABLE dw.fact_subscription_revenue (
    subscription_id         INT             NOT NULL,
    date_key                INT             NOT NULL,
    package_sk              INT             NOT NULL,
    customer_sk             INT             NOT NULL,
    store_sk                INT             NOT NULL,
    package_price           DECIMAL(12,2)   NOT NULL,
    monthly_revenue_usd     DECIMAL(12,2)   NOT NULL,
    status                  VARCHAR(20)     NOT NULL,
    months_active           INT             NOT NULL,
    source_system_code      TINYINT         NOT NULL DEFAULT 0,
    create_timestamp        DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP,
    update_timestamp        DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
```

```
    CONSTRAINT pk_fact_subscription_revenue PRIMARY KEY (subscription_id, date_key),
    CONSTRAINT fk_fsr_date
        FOREIGN KEY (date_key)
        REFERENCES dw.dim_date (date_key),
    CONSTRAINT fk_fsr_package
        FOREIGN KEY (package_sk)
        REFERENCES dw.dim_package (package_sk),
    CONSTRAINT fk_fsr_customer
        FOREIGN KEY (customer_sk)
        REFERENCES dw.dim_customer (customer_sk),
    CONSTRAINT fk_fsr_store
        FOREIGN KEY (store_sk)
        REFERENCES dw.dim_store (store_sk)
);
```

11. Creating stagging area for order header:

```
USE dw;

DROP TABLE IF EXISTS dw.stg_order_header;
CREATE TABLE dw.stg_order_header (
    order_id        INT,
    order_date      VARCHAR(20),
    customer_number INT,
    store_number    INT,
    currency        VARCHAR(5),
    created         VARCHAR(20),
    last_updated    VARCHAR(20),
    purchase_type   VARCHAR(20),
    load_timestamp  DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

12. Creating stagging area for order details:

```
CREATE TABLE dw.stg_order_details (
    order_id        INT,
    line_no         INT,
    product_code    INT,
    qty             INT,
    price           DECIMAL(10,2),
    unit_cost       DECIMAL(10,2),
    load_timestamp  DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

13. Creating stagging area for customers

```
CREATE TABLE dw.stg_customer (
    customer_number     INT,
    customer_type       VARCHAR(5),
    name                VARCHAR(100),
    gender              CHAR(1),
    date_of_birth       VARCHAR(20),
    city                VARCHAR(100),
    state               VARCHAR(10),
    zipcode             INT,
    country             CHAR(2),
    occupation          VARCHAR(100),
    household_income    INT,
    status              VARCHAR(5),
    permission          CHAR(1),
    preferred_channel1  VARCHAR(20),
    preferred_channel2  VARCHAR(20),
    interest1           VARCHAR(50),
    interest2           VARCHAR(50),
    interest3           VARCHAR(50),
    market              VARCHAR(30),
    load_timestamp      DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

14. Creating stagging area for product

```
CREATE TABLE dw.stg_product (
    product_code        INT,
    name                VARCHAR(255),
    description         VARCHAR(255),
    title               VARCHAR(255),
    artist_code         VARCHAR(20),
    product_type_code   CHAR(2),
    format              VARCHAR(20),
    unit_price          DECIMAL(10,2),
    unit_cost           DECIMAL(10,2),
    status              CHAR(2),
    created             VARCHAR(20),
    last_updated        VARCHAR(20),
    load_timestamp      DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

15. Creating stagging area for store

```
CREATE TABLE dw.stg_store (
    store_number    INT,
    store_name      VARCHAR(100),
    store_type      VARCHAR(30),
    address1        VARCHAR(100),
    address2        VARCHAR(100),
    address3        VARCHAR(100),
    city            VARCHAR(100),
    state           VARCHAR(10),
    zipcode         VARCHAR(20),
    country         CHAR(2),
    phone_number    VARCHAR(20),
    web_site        VARCHAR(100),
    region          VARCHAR(50),
    division        VARCHAR(10),
    created         VARCHAR(20),
    last_updated    VARCHAR(20),
    load_timestamp  DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

16. Creating stagging area for currency rate

```
CREATE TABLE dw.stg_currency_rate (
    effective_date  VARCHAR(20),
    currency_code   CHAR(3),
    currency_rate   DECIMAL(10,6),
    created         VARCHAR(20),
    last_updated    VARCHAR(20),
    load_timestamp  DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

17. Creating stagging area for subscription

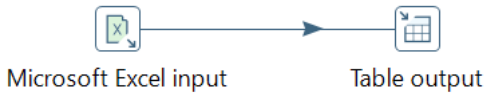
```
CREATE TABLE dw.stg_subscription (
    subscription_id INT,
    customer_id     INT,
    store_id        INT,
    package_id      INT,
    channel_id      INT,
    start_date      INT,
    end_date        INT,
    status          VARCHAR(20),
    load_timestamp  DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

18. Creating stagging area for package

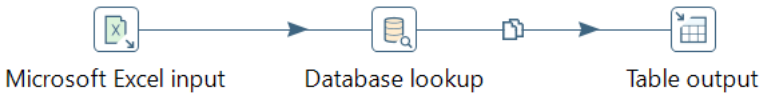
```
CREATE TABLE dw.stg_package (
    package_id      INT,
    name            VARCHAR(100),
    description     VARCHAR(255),
    package_type    CHAR(2),
    package_price   DECIMAL(10,2),
    load_timestamp  DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

Transformations used

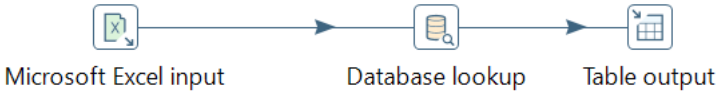
1. Transformation 1: product_category



2. Transformation 2: product type

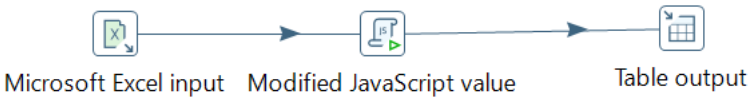


3. Transformation 3: product

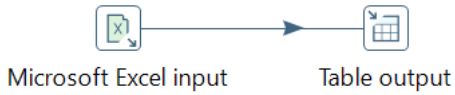


4. Transformation 4: currency

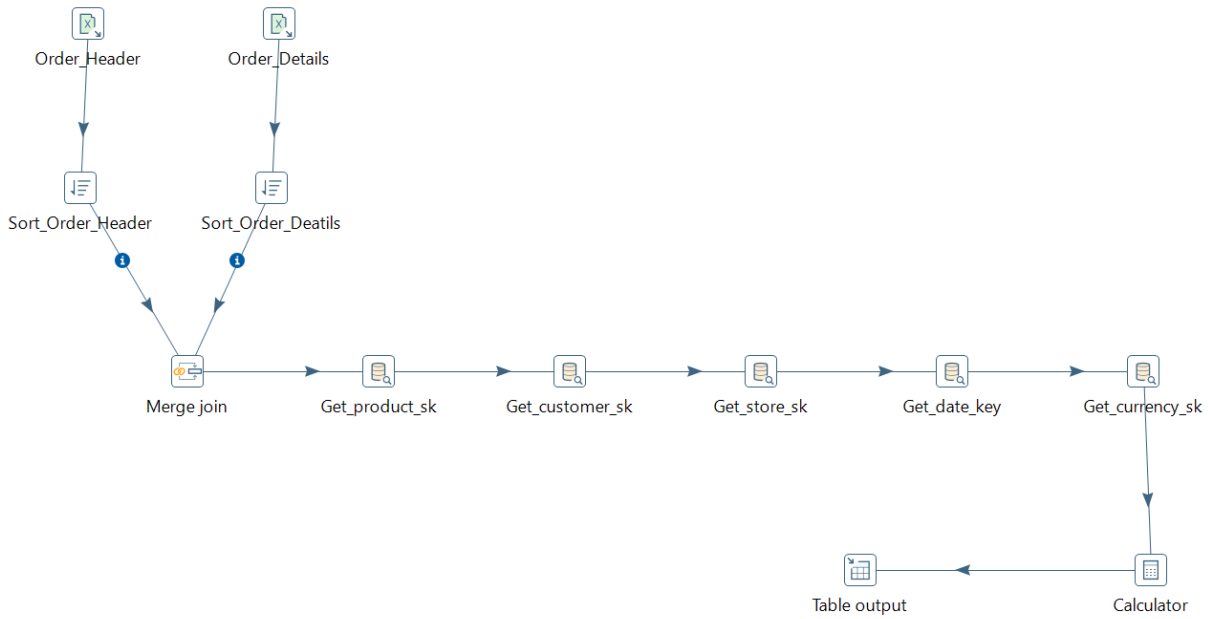
5. Transformation 5: customer



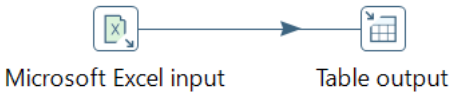
6. Transformation 6: store



7. Transformation 7: product_sales



8. Transformation 8: package



9. Transformation 9: Subscription_revenue